

Day 5 — Lab 12 (Adapted)

Agentic SOC Platform

Click-by-Click Build & Handover Guide

Kali (VirtualBox) → Docker Compose → ASP | Wazuh (webhook push) | Gemini (Google AI Studio)

Verified build — every command below is the one that actually worked, not the vendor default.

At a glance	Value
End result	Windows endpoint → Wazuh → ASP webhook → Case with a live Gemini verdict
Host	Kali VM on VirtualBox (~8 GB RAM / ~24 GB disk). NAT for internet + Host-Only for the agent.
LLM	Google AI Studio (Gemini) via OpenAI-compatible endpoint — no local model.
The 3 things that block a Case	(1) Module loaded; (2) webhook path = /api/webhook/kibana/ ; (3) stream name matches.
Safety net	VirtualBox snapshot 'Before-Lab12-ASP' = one-click undo.
Handover files	custom-asp + wazuh_alerts_module.py (both shipped with this guide)

Contents

0. Read First — Scope & The Gotchas We Hit

SCOPE — “Lab 12, adapted”

The team tracker lists Lab 12 = Agentic SOC Platform with no original handbook. This guide is faithful to the live GitHub repo (FunnyWolf/agentive-soc-platform, v0.5.2) with three deliberate swaps: host = Kali on VirtualBox, LLM = Gemini, SIEM = Wazuh via webhook. Wazuh is NOT natively supported (ASP only speaks Splunk + ELK/Kibana natively), so we wire Wazuh in as a custom webhook source that masquerades as a Kibana webhook.

This document was rewritten AFTER a full working build. The vendor's own docs drifted from the shipped code in several places, so the steps below are corrected to what actually worked. The most important corrections:

Area	Vendor default said...	What actually worked
Docker on Kali	<code>curl get.docker.com sh</code>	Kali has no Docker repo → apt install docker.io + drop the Compose v2 plugin manually (Step 1).
Bundle download	<code>latest/download/asp-compose.tar.gz</code>	That 404s (asset is versioned). Fetch the real asset via the GitHub API (Step 2).
HTTPS port	443	Wazuh dashboard already owns 443 → run ASP on 8443 via <code>.env</code> (Step 4).
Webhook path	<code>/webhook/<stream></code>	Returns 405. Real route is <code>/api/webhook/kibana/</code> (Step 9).
Webhook auth	token in <code>.env</code>	No token in <code>.env</code> . Generate a per-user API Key in the ASP UI (Step 9).
Module API	<code>self.create_alert_with_context(...)</code>	BaseModule has no helpers. Import the free function and pass the exact model fields (Step 7).

TAKE THE SNAPSHOT NOW

VirtualBox → select the Kali VM → Take Snapshot → name it 'Before-Lab12-ASP'. If anything goes sideways, Restore Snapshot = total clean rollback. This is your undo button for the whole lab.

SECRETS — keep these OUT of screenshots

Two secrets are created during this lab: the Gemini API key (entered in the ASP UI) and the ASP webhook API key (put in `ossec.conf`). Blur/crop both in any screenshot, and revoke/rotate both at teardown (Part D).

What you need in front of you

- Kali VM on VirtualBox, ~8 GB RAM, ~24 GB free disk, internet access via NAT.
- Wazuh Server (manager + indexer + dashboard) installed on the Kali box — see Appendix A if you need to install it.
- A Windows machine to run the agent (the host laptop is fine).
- A Google account (for a free Google AI Studio API key).
- The two files shipped with this guide: `custom-asp` and `wazuh_alerts_module.py`.

PART A — Stand Up ASP on Kali

Step 1 — Install Docker on Kali (the corrected way)

WHY NOT `get.docker.com`

Kali reports its codename as 'kali-rolling', which Docker's convenience script tries to map to a Debian repo that doesn't exist → apt update then errors. Use Kali's own `docker.io` package and add the Compose v2 plugin by hand. The `docker-compose-v2` apt package is NOT reliably available on Kali, which is why we drop the plugin binary directly.

Run one at a time:

```
sudo apt update
sudo apt install -y docker.io
```

Add the Docker Compose v2 plugin system-wide (so it also works under sudo, which `init.sh` needs):

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-
x86_64 -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
```

Start Docker and verify (run the two version checks SEPARATELY):

```
sudo systemctl enable --now docker
docker --version
docker compose version
```

CHECKPOINT

Both print a version, and 'docker compose version' (v2, space) works. A 'zsh: corrupt history file' line is cosmetic — ignore it. If the install pops a debconf prompt about `/var/lib/docker`, choose <No> (keep data).

```

kali@kali: ~
Session Actions Edit View Help

Preconfiguring packages ...
(Reading database... 598617 files and directories currently installed.)
Preparing to unpack .../docker-cli_28.5.2+dfsg4-3_amd64.deb...
Unpacking docker-cli (28.5.2+dfsg4-3) over (28.5.2+dfsg3-2)...
Preparing to unpack .../docker.io_28.5.2+dfsg4-3_amd64.deb...
Unpacking docker.io (28.5.2+dfsg4-3) over (28.5.2+dfsg3-2)...
Setting up docker-cli (28.5.2+dfsg4-3)...
Setting up docker.io (28.5.2+dfsg4-3)...
Processing triggers for kali-menu (2026.2.2)...
Processing triggers for man-db (2.13.1-1)...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...
systemctl restart packagekit.service wazuh-indexer.service
Service restarts being deferred:
systemctl restart NetworkManager.service
systemctl restart docker.service

No containers need to be restarted.

User sessions running outdated binaries:
kali @ user service: xdg-desktop-portal.service[4829]

No VM guests are running outdated hypervisor (qemu) binaries on this host.

(kali@kali)-[~]
$ sudo mkdir -p /usr/local/lib/docker/cli-plugins

(kali@kali)-[~]
$ sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 -o /usr
/local/lib/docker/cli-plugins/docker-compose
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed

  0     0    0     0    0     0      0      0      0     0
  0     0    0     0    0     0      0      0      0     0
100 47.07M 100 47.07M    0     0  13.25M      0  00:03  00:03  14.65M

(kali@kali)-[~]
$ sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

(kali@kali)-[~]
$ docker --version and docker compose version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

(kali@kali)-[~]
$

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed

(kali@kali)-[~]
$ docker compose version
Docker Compose version v5.4.0

(kali@kali)-[~]
$

```

Step 2 — Download the ASP bundle (via GitHub API)

WHY NOT the 'latest/download' URL

The release asset is versioned (asp-compose-0.5.2.tar.gz), so the generic latest/download/asp-compose.tar.gz URL returns 404. Ask the GitHub API for the real asset URL instead — this also future-proofs against version bumps.

```
cd ~
ASSET_URL=$(curl -s https://api.github.com/repos/FunnyWolf/agentik-soc-
platform/releases/latest | grep "browser_download_url.*asp-compose.*tar.gz" | cut -d '"' -f 4)
echo "$ASSET_URL"          # should print a real ../v0.5.x/asp-compose-0.5.x.tar.gz URL
curl -fL -o asp-compose.tar.gz "$ASSET_URL"
tar -xzf asp-compose.tar.gz && rm asp-compose.tar.gz && cd asp-compose
ls -la                     # expect scripts/ certs/ custom/ .env.example compose.yaml
```

DON'T RENAME THE FOLDER

The folder name becomes the Compose project + data-volume prefix. Rename it later and you orphan all your data. Leave it as asp-compose.

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ ASSET_URL=$(curl -s https://api.github.com/repos/FunnyWolf/agent-soc-platform/releases/latest | grep "browser_download_url.*asp-compose.*tar.gz" | cut -d '"' -f 4)  
  
(kali@kali)-[~]  
$ echo "$ASSET_URL"  
https://github.com/FunnyWolf/agent-soc-platform/releases/download/v0.5.2/asp-compose-0.5.2.tar.gz  
  
(kali@kali)-[~]  
$ curl -fL -o asp-compose.tar.gz "$ASSET_URL"  
% Total % Received % Xferd Average Speed Time Time Time Current  
Dload Upload Total Spent Left Speed  
0 0 0 0 0 0 0 0  
100 8783 100 8783 0 0 0 12867 0 0  
  
(kali@kali)-[~]  
$ tar -xzf asp-compose.tar.gz 66 rm asp-compose.tar.gz  
  
(kali@kali)-[~]  
$ ls -la  
total 500  
drwxr-xr-x 27 kali kali 4096 Aug 4 16:14 .  
drwxr-xr-x 4 root root 4096 Apr 18 2025 ..  
drwxr-xr-x 6 kali kali 4096 Jul 27 23:01 asp-compose  
t  
iginal  
g  
3  
i  
4096 Apr 22 23:11 DOWNLOADS  
→ .face  
save.1  
ity  
shes.txt  
a
```

```

(kali㉿kali)-[~]
$ cd asp-compose

(kali㉿kali)-[~/asp-compose]
$ ls -la
total 52
drwxr-xr-x  6 kali kali 4096 Jul 27 23:01 .
drwx----- 27 kali kali 4096 Aug  4 16:14 ..
drwxr-xr-x  2 kali kali 4096 Jul 27 23:01 certs
-rw-r--r--  1 kali kali 5021 Jul 27 23:01 compose.yaml
drwxr-xr-x  5 kali kali 4096 Jul 27 23:01 custom
-rw-r--r--  1 kali kali 1673 Jul 27 23:01 .env.example
drwxr-xr-x  3 kali kali 4096 Jul 27 23:01 logs
-rw-r--r--  1 kali kali 7071 Jul 27 23:01 README.md
-rw-r--r--  1 kali kali 6901 Jul 27 23:01 README.zh.md
drwxr-xr-x  2 kali kali 4096 Jul 27 23:01 scripts

(kali㉿kali)-[~/asp-compose]
$ █

```

Step 3 — Initialize (generates secrets, pulls images, migrates, starts)

```
./scripts/init.sh
```

THIS IS THE LONG STEP

init.sh writes a .env full of random secrets, pulls ~5 images from ghcr.io (needs outbound HTTPS, watch your 24 GB disk), runs DB migrations, and starts everything. It ends by trying to bring up all containers.

On this build the frontend failed on its FIRST start with:

```
Error ... failed to bind host port for 0.0.0.0:443 ... address already in use
```

That is expected here because the Wazuh dashboard already owns 443. Fix it in Step 4 — do NOT re-run init.sh.


```

kali@kali: ~/asp-compose
Session Actions Edit View Help

(kali@kali)~[~/asp-compose]
$ ./scripts/init.sh
Generated .env with random service secrets. Review .env before exposing the deployment.
[+] pull 63/63
✓ Image redis/redis-stack:latest Pulled 179.5s
✓ Image rustfs/rustfs:latest Pulled 111.3s
✓ Image postgres:17 Pulled 81.9s
✓ Image ghcr.io/funnywolf/agenttic-soc-platform/asp-backend:0.5.2 Pulled 60.6s
✓ Image ghcr.io/funnywolf/agenttic-soc-platform/asp-frontend:0.5.2 Pulled 79.7s
[+] 9/9t 9/99
✓ Network asp-compose_default Created 0.1s
✓ Volume asp-compose_custom-python-packages Created 0.0s
✓ Volume asp-compose_postgres-data Created 0.0s
✓ Volume asp-compose_redis-data Created 0.0s
✓ Volume asp-compose_rustfs-data Created 0.0s
✓ Volume asp-compose_static-files Created 0.0s
✓ Container asp-compose-rustfs-1 Started 0.6s
✓ Container asp-compose-redis-stack-1 Started 0.6s
✓ Container asp-compose-postgres-1 Started 0.6s
Container asp-compose-postgres-1 Waiting
Container asp-compose-redis-stack-1 Waiting
Container asp-compose-postgres-1 Healthy
Container asp-compose-redis-stack-1 Healthy
Container asp-compose-asp-migrate-run-0250694db87a Creating
Container asp-compose-asp-migrate-run-0250694db87a Created
Operations to perform:
Apply all migrations: accounts, agentic, alerts, artifacts, attachments, audit, auth, cases, comments, common,
contenttypes, enrichments, inbox, knowledge, playbooks, preferences, sessions, settings
Running migrations:
Applying contenttypes.0001_initial ... OK
Applying contenttypes.0002_remove_content_type_name ... OK
Applying auth.0001_initial ... OK
Applying auth.0002_alter_permission_name_max_length ... OK
Applying auth.0003_alter_user_email_max_length ... OK
Applying auth.0004_alter_user_username_opts ... OK
Applying auth.0005_alter_user_last_login_null ... OK
Applying auth.0006_require_contenttypes_0002 ... OK
Applying auth.0007_alter_validators_add_error_messages ... OK
Applying auth.0008_alter_user_username_max_length ... OK
Applying auth.0009_alter_user_last_name_max_length ... OK
Applying auth.0010_alter_group_name_max_length ... OK
Applying auth.0011_update_proxy_permissions ... OK
Applying auth.0012_alter_user_first_name_max_length ... OK
Applying accounts.0001_initial ... OK
Applying attachments.0001_initial ... OK
Applying accounts.0002_initial ... OK
Applying cases.0001_initial ... OK
Applying agentic.0001_initial ... OK
Applying cases.0002_case_case_created_id_idx ... OK

```

Step 4 — Move ASP to port 8443 (443 is taken by Wazuh)

Confirm who owns 443 (it will be the Wazuh dashboard, shown as 'node'):

```
sudo ss -tlnp | grep ':443'
```

The compose file parameterises the port, so we just set it in .env — no YAML editing:

```
echo "ASP_HTTPS_PORT=8443" >> .env
```

```
grep ASP_HTTPS_PORT .env      # last line wins; 8443 is what takes effect
```

Bring the stack up on the new port (only starts what's missing — no re-pull):

```
docker compose up -d
```

REMEMBER THIS

From now on ASP lives at `https://<kali-ip>:8443/` (Wazuh keeps 443). Both run side by side. If your box is older/smaller, this same trick works for any port conflict — just pick a free port.

```

(kali㉿kali)-[~/asp-compose]
$ sudo ss -tlnp | grep ':443'
[sudo] password for kali:
LISTEN 0      511          0.0.0.0:*        users:((("node",pid=543,fd=19))

(kali㉿kali)-[~/asp-compose]
$ grep -n "443" compose.yaml
28: - "${ASP_BIND:-0.0.0.0}:${ASP_HTTPS_PORT:-443}:443"

(kali㉿kali)-[~/asp-compose]
$ ls -la compose.override.yaml 2>/dev/null 86 grep -n "443\|HTTPS\|PORT" compose.override.yaml .env 2>/dev/nul
l

(kali㉿kali)-[~/asp-compose]
$

(kali㉿kali)-[~/asp-compose]
$ echo "ASP_HTTPS_PORT=8443" >> .env

(kali㉿kali)-[~/asp-compose]
$ grep ASP_HTTPS_PORT .env
ASP_HTTPS_PORT=443
ASP_HTTPS_PORT=8443

```

Step 5 — Verify the stack is healthy

```

docker compose ps
./scripts/doctor.sh

```

You want every service Up/healthy. The frontend line should show 0.0.0.0:8443->443/tcp.

HARMLESS ERROR

doctor.sh may end with a traceback: "No module named 'apps.agentic.services.custom_scripts'". That is only the custom-scripts self-check firing on an empty custom/ folder — the platform is fine (it appears AFTER all the healthy/PONG/OK lines). It disappears once the Module is loaded in Step 7.

```

kali@kali: ~/asp-compose
Session Actions Edit View Help

(kali@kali)-[~/asp-compose]
$ docker compose up -d
[+] up 13/13
✓ Container asp-compose-postgres-1           Healthy      11.6s
✓ Container asp-compose-redis-stack-1         Healthy      11.6s
✓ Container asp-compose-rustfs-1              Running      0.0s
✓ Container asp-compose-asp-custom-deps-1      Started      11.3s
✓ Container asp-compose-asp-asgi-1             Healthy      45.7s
✓ Container asp-compose-asp-migrate-1          Started      12.0s
✓ Container asp-compose-asp-worker-module-1    Started      13.0s
✓ Container asp-compose-asp-worker-case-analysis-1 Started      12.7s
✓ Container asp-compose-asp-web-1              Healthy      45.7s
✓ Container asp-compose-asp-worker-playbook-1  Started      13.3s
✓ Container asp-compose-asp-worker-dashboard-cache-1 Started      13.3s
✓ Container asp-compose-asp-worker-elk-action-1 Started      13.2s
✓ Container asp-compose-asp-frontend-1         Started      44.5s

(kali@kali)-[~/asp-compose]
$ ./scripts/doctor.sh
NAME                SERVICE                IMAGE                STATUS                PORTS                COMMAND
asp-compose-asp-asgi-1    asp-asgi                ghcr.io/funnywolf/agenti...asp-backend:0.5.2    "uvicorn
asp-asgi:ap..."        2 minutes ago          Up 2 minutes (healthy)    8000-8001/tcp
asp-compose-asp-frontend-1  asp-frontend            ghcr.io/funnywolf/agenti...asp-frontend:0.5.2    "/docker-
entrypoint..."        2 minutes ago          Up About a minute (healthy)  80/tcp, 0.0.0.0:8443
→443/tcp
asp-compose-asp-web-1      asp-web                  ghcr.io/funnywolf/agenti...asp-backend:0.5.2    "gunicorn
asp.wsgi:a..."          2 minutes ago          Up 2 minutes (healthy)    8000-8001/tcp
asp-compose-asp-worker-case-analysis-1  asp-worker-case-analysis  ghcr.io/funnywolf/agenti...asp-backend:0.5.2    "python m
anage.py ru..."        2 minutes ago          Up 2 minutes              8000-8001/tcp
asp-compose-asp-worker-dashboard-cache-1  asp-worker-dashboard-cache  ghcr.io/funnywolf/agenti...asp-backend:0.5.2    "python m
anage.py ru..."        2 minutes ago          Up 2 minutes              8000-8001/tcp
asp-compose-asp-worker-elk-action-1      asp-worker-elk-action      ghcr.io/funnywolf/agenti...asp-backend:0.5.2    "python m
anage.py ru..."        2 minutes ago          Up 2 minutes              8000-8001/tcp
asp-compose-asp-worker-module-1          asp-worker-module          ghcr.io/funnywolf/agenti...asp-backend:0.5.2    "python m
anage.py ru..."        2 minutes ago          Up 2 minutes              8000-8001/tcp
asp-compose-asp-worker-playbook-1        asp-worker-playbook        ghcr.io/funnywolf/agenti...asp-backend:0.5.2    "python m
anage.py ru..."        2 minutes ago          Up 2 minutes              8000-8001/tcp
asp-compose-postgres-1      postgres:17              postgres:17              Up 12 minutes (healthy)    5432/tcp
ntrypoint.s..."        12 minutes ago          Up 12 minutes (healthy)
asp-compose-redis-stack-1    redis/redis-stack:latest  redis/redis-stack:latest  Up 12 minutes (healthy)    6379/tcp, 0.0.0.0:80
int.sh"                  12 minutes ago          Up 12 minutes (healthy)
01→8001/tcp
asp-compose-rustfs-1        rustfs/rustfs:latest      rustfs/rustfs:latest      Up 12 minutes              9000/tcp, 0.0.0.0:90
int.sh rust..."          12 minutes ago          Up 12 minutes
01→9001/tcp
/var/run/postgresql:5432 - accepting connections
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
PONG
System check identified no issues (0 silenced).
33 objects imported automatically (use -v 2 for details).

```

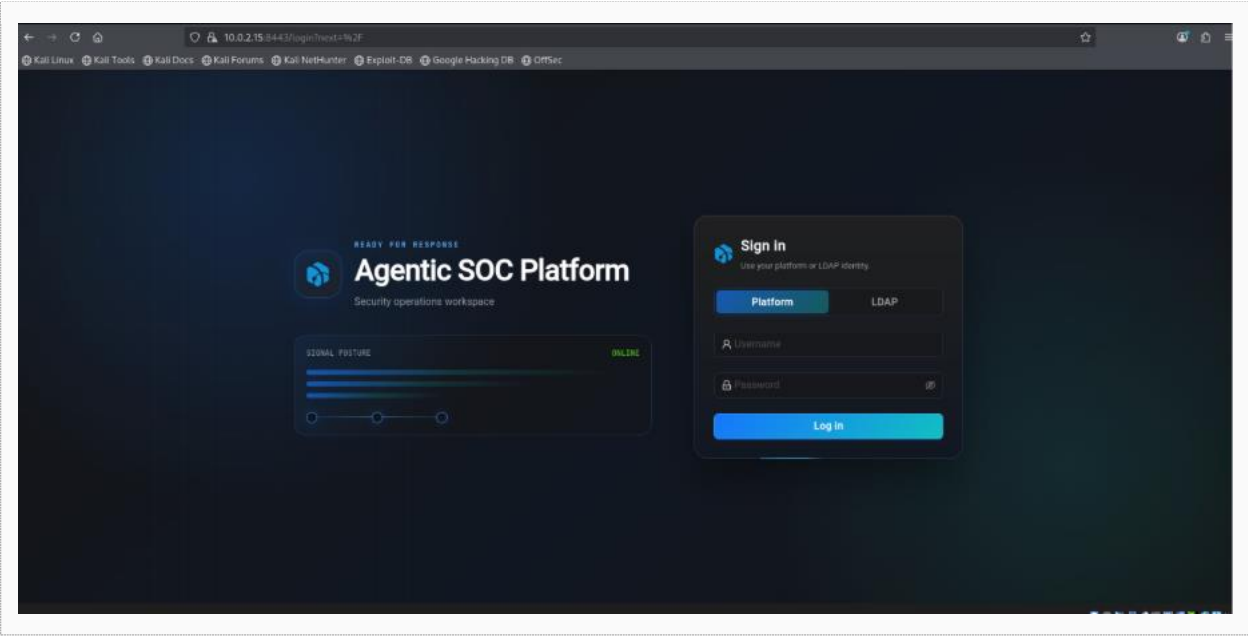
Step 6 — First login & create your admin

Open the UI (accept the one-time self-signed certificate warning):

```

# on Kali:    https://localhost:8443/
# from LAN:   https://<kali-ip>:8443/    (find <kali-ip> with: ip a )

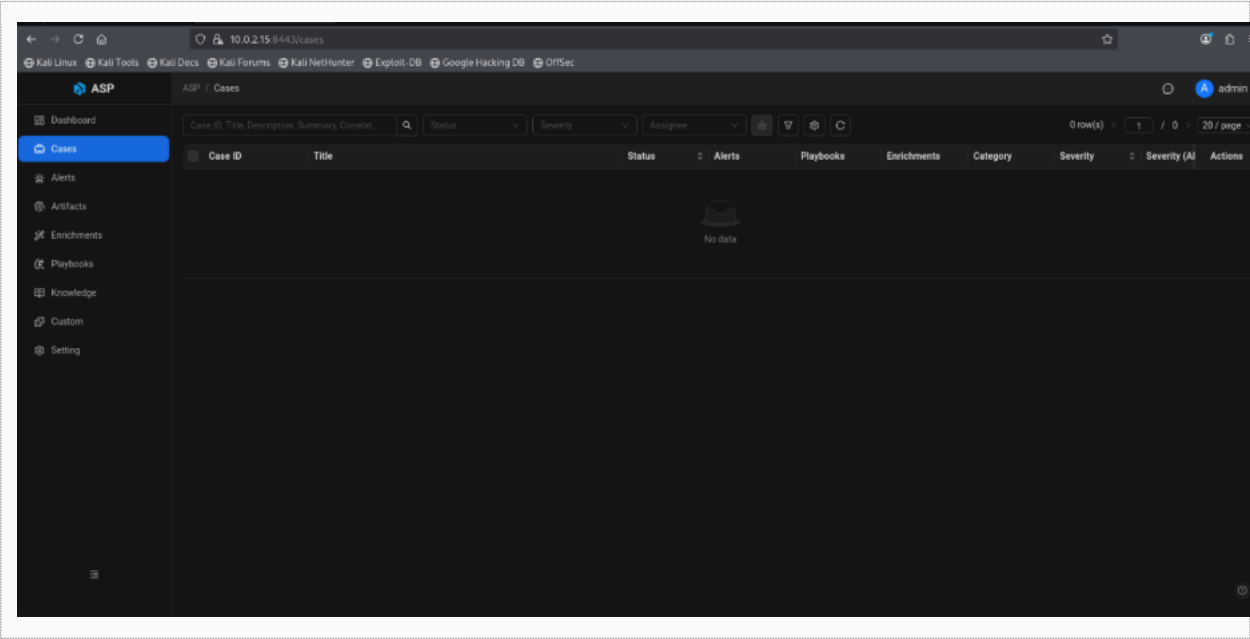
```



Create the superuser from the terminal, then log in with it on the Platform tab:

```
docker compose exec asp-web python manage.py createsuperuser
```

Enter username / email / a strong password (the password field shows nothing as you type — normal). You want 'Superuser created successfully.'



Step 7 — Wire up Gemini as the LLM

- 1. Go to aistudio.google.com → Get API key → Create API key. Copy it somewhere safe (NOT into a screenshot).
- 2. In ASP: Setting → LLM Providers → + (add) → fill the form with the values below.

Field	Value
Provider type / Name	OpenAI-compatible

Field	Value
Base URL	https://generativelanguage.googleapis.com/v1beta/openai/
Model	gemini-3.1-flash-lite (use the EXACT lowercase-hyphen API id, not the display name)
API key	<YOUR_GOOGLE_AI_STUDIO_KEY> (paste in UI only)
Proxy	leave blank (the greyed 127.0.0.1:7890 is placeholder text)
Tags	fast, powerful, function_calling, structured_output
Enabled	Yes (at least one provider MUST be enabled)

3. Click Test. If it fails once with a connection/timeout error, click Test again — see the note below. When it succeeds, click Save.

WHY THE MODEL FIELD BITES PEOPLE

The Model field needs the exact API id (e.g. gemini-3.1-flash-lite), NOT the friendly name 'Gemini 3.1 Flash Lite'. A name with spaces/capitals fails the Test.

WHY 'TEST' SOMETIMES TIMES OUT THEN WORKS

ASP has a short read-timeout on the Test call and POSTs to <base_url>/chat/completions. gemini-3.1-flash-lite can be slow on the first token, so the first Test may return 'connection error' (really an httpx ReadTimeout) while the next one returns HTTP 200. The config is correct — just retry. We chose 3.1-flash-lite for its higher rate/quota headroom. If verdicts are ever slow under load, lower the model's thinking budget or add gemini-2.5-flash as a second 'fast'-tagged provider.

DATA EGRESS

A remote LLM means alert/IOC text leaves your network to Google. Use synthetic/lab data only — never real client data. (Claude Code is an optional Harness Agent integration and is intentionally skipped; all AI runs on this Gemini provider.)

The image shows two screenshots of the Google AI Studio interface. The top screenshot is the 'Playground' view, and the bottom screenshot is the 'API Keys' view.

Top Screenshot: Playground

- Header:** Google AI Studio, Playground
- Left Sidebar:** EXPLORE (Playground, History), BUILD (New app, My apps, Gallery), MANAGE (Dashboard, Documentation).
- Main Content:** 'Explore Google models' section with cards for Featured, Code and Chat, Image Generation, Video Generation, Speech and Music, and Real-time.
- Right Sidebar:** Run settings (Get code, X), Gemini 3.1 Pro Preview, System instructions, Temperature slider, Thinking level dropdown, Tools, Structured outputs, Code execution, Function calling, Grounding with Google Search, and Grounding with Google Maps.
- Bottom:** 'Start building' section with a prompt input field and a 'Tools' button.

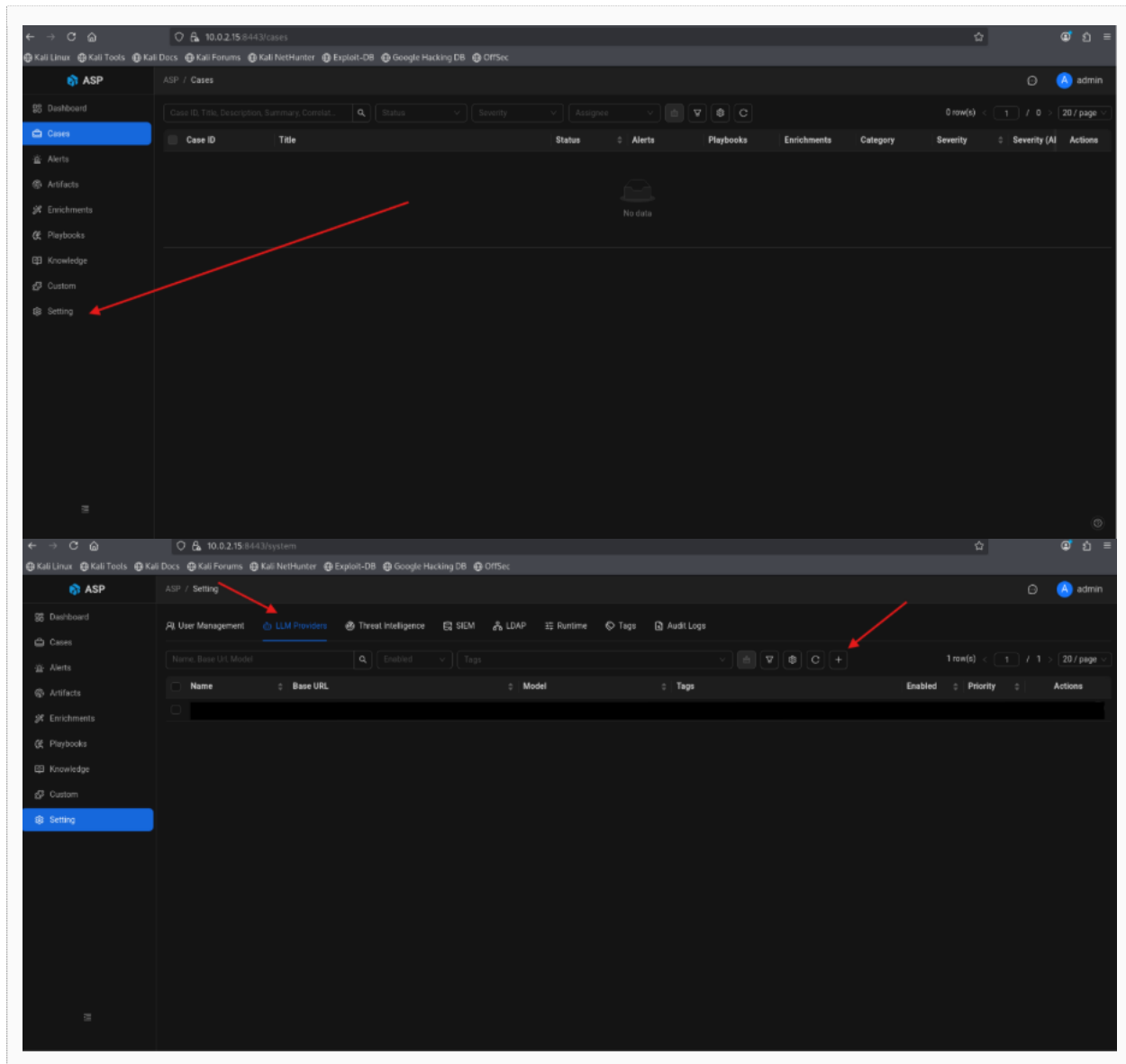
Bottom Screenshot: API Keys

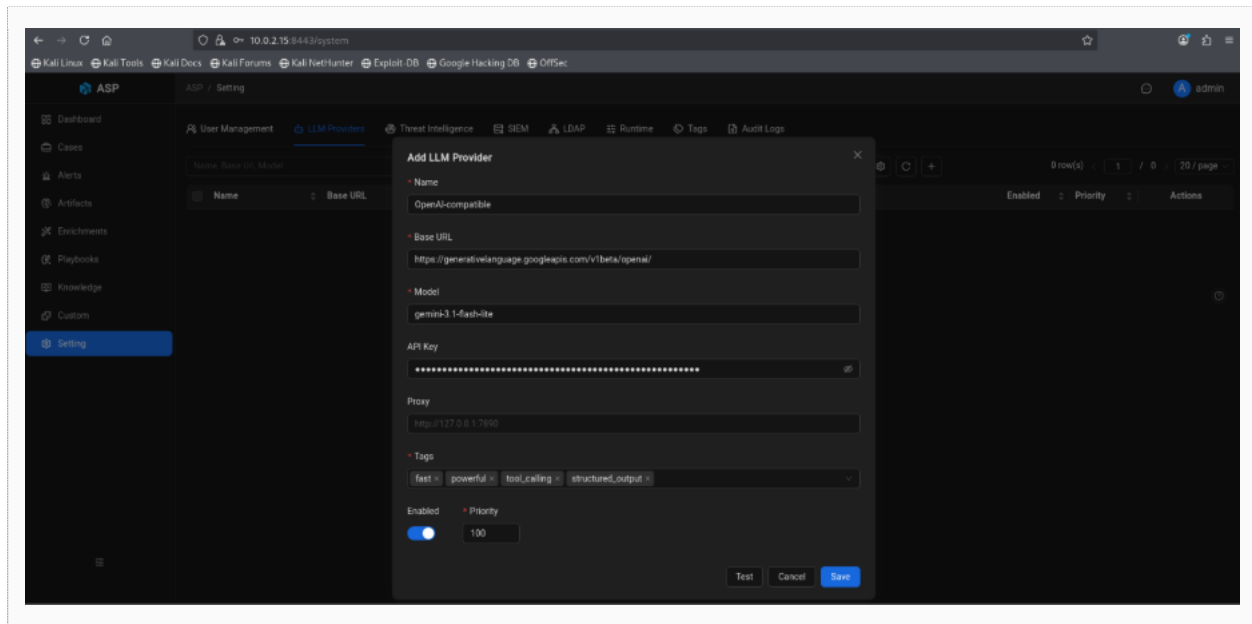
- Header:** Google AI Studio, API Keys
- Left Sidebar:** PROJECT (API Keys, Projects), USAGE & BILLING (Usage, Rate Limit, Spend, Billing), OBSERVE (Logs and Datasets), Changelog.
- Main Content:** 'API Keys' table with columns: Key, Project, Created, Billing Tier. It shows one key: '1B4A n8n_test_key' for 'n8n test project'.
- Right:** 'API quickstart' and 'Create API key' buttons.
- Bottom:** 'Can't find your API keys here?' message with a link to 'Learn more' and an 'Import projects' button.

API Key details modal:

- Title:** API key details
- Content:** For details on using auth keys with Gemini API, see our API key documentation.
- Fields:** API Key (1B4A), Name, Project name, Project number.
- Buttons:** Delete key, Copy cURL quickstart, Copy key.

Now, go back to the ASP and follow the screenshots





PART B — Teach ASP to Understand Wazuh

THE MENTAL MODEL (read once, save an hour)

ASP does not know the word 'Wazuh'. It only knows: a Redis Stream (a named inbox) + a Module (code that reads that inbox and makes Cases). Our webhook drops alerts into the stream 'Wazuh-Alerts'; our Module reads 'Wazuh-Alerts'. The stream name written by the webhook and the STREAM_NAME in the Module MUST be identical, or alerts pile up unread and no Case ever appears.

Step 8 — Install the Wazuh-Alerts Module (ASP side)

Copy the shipped wazuh_alerts_module.py into the release package's custom modules folder. If you dragged it onto the Kali desktop, copy from there:

```
cd ~/asp-compose && mkdir -p custom/modules
cp ~/Desktop/wazuh_alerts_module.py custom/modules/ # adjust source path as needed
ls -la custom/modules/ && head -5 custom/modules/wazuh_alerts_module.py
```

Reload it in the UI: Custom → Modules → Refresh / Validate. Confirm a row appears:

Column	Expected
Module	Wazuh Alerts
Stream	Wazuh-Alerts (this MUST match the webhook)
Threads	1
Path	/app/custom/modules/wazuh_alerts_module.py

WHAT THIS MODULE DOES (and why it's written the way it is)

It maps each Wazuh alert onto ASP's real SIRP model and calls the standalone function create_alert_with_context(...) with schedule_analysis=True (which triggers Gemini). Hard-won facts baked into it: BaseModule exposes ONLY NAME/DESC/STREAM_NAME/THREAD_NUM/run — there are no helper methods; the Alert model uses 'desc' (not 'description'), plus first_seen_time/last_seen_time/rule_id/rule_name/raw_data; artifacts go through get_or_create_artifact(*, name, type, role, value) where 'name' is an OCSF category label (e.g. 'Source IP', 'Hostname', 'User Name'), 'type'/'role' are fixed enums, and the real value goes in 'value'. The description is enriched (rule id+level, user, IPs, Windows Event ID, MITRE, raw log) so Gemini returns a real verdict instead of 'Insufficient Data'. The shipped file is authoritative — don't retype it.

Column

Expected

```

kali@kali: ~/asp-compose
Session Actions Edit View Help
asp-web-1 | File "/app/.venv/lib/python3.14/site-packages/httpx/_client.py", line 914, in send
asp-web-1 |     response = self._send_handling_auth(
asp-web-1 |         request,
asp-web-1 |         ... <2 lines> ...
asp-web-1 |         history=[],
asp-web-1 |     )
asp-web-1 | File "/app/.venv/lib/python3.14/site-packages/httpx/_client.py", line 942, in _send_handling_auth
asp-web-1 |     response = self._send_handling_redirects(
asp-web-1 |         request,
asp-web-1 |         follow_redirects=follow_redirects,
asp-web-1 |         history=history,
asp-web-1 |     )
asp-web-1 | File "/app/.venv/lib/python3.14/site-packages/httpx/_client.py", line 979, in _send_handling_redirects
asp-web-1 |     response = self._send_single_request(request)
asp-web-1 | File "/app/.venv/lib/python3.14/site-packages/httpx/_client.py", line 1014, in _send_single_request
asp-web-1 |     response = transport.handle_request(request)
asp-web-1 | File "/app/.venv/lib/python3.14/site-packages/httpx/_transports/default.py", line 249, in handle_request
    with map_httpcore_exceptions():
asp-web-1 |         ~~~~~^
asp-web-1 | File "/usr/local/lib/python3.14/contextlib.py", line 162, in __exit__
asp-web-1 |     self.gen.throw(value)
asp-web-1 |         ~~~~~^
asp-web-1 | File "/app/.venv/lib/python3.14/site-packages/httpx/_transports/default.py", line 118, in map_httpcore_exceptions
    raise mapped_exc(message) from exc
asp-web-1 | httpx.ReadTimeout: The read operation timed out
asp-web-1 | 2026-08-04 16:17:22,186 INFO [httpx:1025] HTTP Request: POST https://generativelanguage.googleapis.com/v1
asp-web-1 | beta/openai/chat/completions "HTTP/1.1 200 OK"
asp-web-1 | 2026-08-04 16:17:44,484 INFO [httpx:1025] HTTP Request: POST https://generativelanguage.googleapis.com/v1
asp-web-1 | beta/openai/chat/completions "HTTP/1.1 200 OK"
^C

(kali@kali)~/asp-compose
$ mkdir -p custom/modules

(kali@kali)~/asp-compose
$ cp ~/Desktop/wazuh_alerts_module.py custom/modules/

(kali@kali)~/asp-compose
$ ls -la custom/modules/ 86 head -5 custom/modules/wazuh_alerts_module.py
total 12
drwxr-xr-x 2 kali kali 4096 Aug  4 17:28 .
drwxr-xr-x 5 kali kali 4096 Jul 27 23:01 ..
-rw-r--r-- 1 kali kali  0 Jul 27 23:01 .gitkeep
-rw-rw-r-- 1 kali kali 3916 Aug  4 17:28 wazuh_alerts_module.py
# wazuh_alerts_module.py
# ASP custom Module: consumes the "Wazuh-Alerts" Redis Stream (fed by the
# custom-asp Wazuh webhook integration) and turns each alert into a
# Case / Alert / Artifact in the SIRP, then lets Gemini analyse it.
#

```

PART B.5 — Connect the Windows Agent (networking)

PS: You can find the full documentation on the official Wazuh Website

THE NETWORKING PROBLEM, PLAINLY

Kali runs on VirtualBox NAT (its IP is 10.0.2.15). NAT gives Kali internet (needed for Gemini) but the Windows host CANNOT reach 10.0.2.15. So the agent needs a second path. Solution: keep NAT for internet AND add a Host-Only adapter so Kali also gets a 192.168.56.x address the Windows host can reach. You do NOT replace NAT — you add a second NIC.

Step 9 — Add a Host-Only adapter to Kali

4. Snapshot first (cheap insurance): VirtualBox → Kali → Take Snapshot.
5. VirtualBox → Kali → Settings → Network. Leave Adapter 1 = NAT.
6. Adapter 2 tab → tick 'Enable Network Adapter' → Attached to: Host-only Adapter → pick 'VirtualBox Host-Only Ethernet Adapter' (create one via File → Tools → Network Manager if the list is empty) → OK.
7. Boot Kali, then find the new address:

```
ip -4 addr show | grep inet
```

You should now see BOTH: eth0 = 10.0.2.15 (NAT, internet) and eth1 = 192.168.56.x (Host-Only). Write down the 192.168.56.x — that is the manager IP the Windows agent will use.

AUTO-RECOVERY

Docker, ASP containers, and Wazuh are all enabled at boot, so they come back on their own after the reboot. Give it 2–3 minutes. If the Wazuh dashboard briefly says 'No API available', that's just startup timing — wait, or run: `sudo systemctl restart wazuh-manager`. ASP self-serves on the new IP at `https://<ip>:8443/` (nothing to reconfigure).

Step 10 — Deploy the Windows agent (point it at the Host-Only IP)

On the Windows machine, open PowerShell as Administrator. If an old agent exists, remove it first:

```
& "C:\Program Files (x86)\ossec-agent\wazuh-control.exe" stop
msiexec.exe /x $env:tmp\wazuh-agent /q
Remove-Item -Recurse -Force "C:\Program Files (x86)\ossec-agent"
```

Install the agent pointing WAZUH_MANAGER at the Host-Only IP from Step 9 (replace 192.168.56.105 with yours):

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.12.0-1.msi \
-OutFile $env:tmp\wazuh-agent;
msiexec.exe /i $env:tmp\wazuh-agent /q \
WAZUH_MANAGER='192.168.56.105' WAZUH_AGENT_GROUP='default' WAZUH_AGENT_NAME='win-lab-test'
Start-Service WazuhSvc
```

MULTI-MACHINE NOTE (for whoever reproduces this)

We built this on a single laptop: Kali (manager + ASP) in a VM, Windows agent on the host. To onboard MORE devices, install the Wazuh agent on each and set WAZUH_MANAGER to a manager IP that device can actually reach (Host-Only 192.168.56.x for same-host VMs, or the manager's LAN IP if bridged). Everything downstream (custom-asp, the Module, Gemini) is unchanged — more agents just means more alerts into the same pipeline.

Confirm on Kali that the agent is Active:

```
sudo /var/ossec/bin/agent_control -l
```

```

kali@kali: ~/asp-co
Session Actions Edit View Help

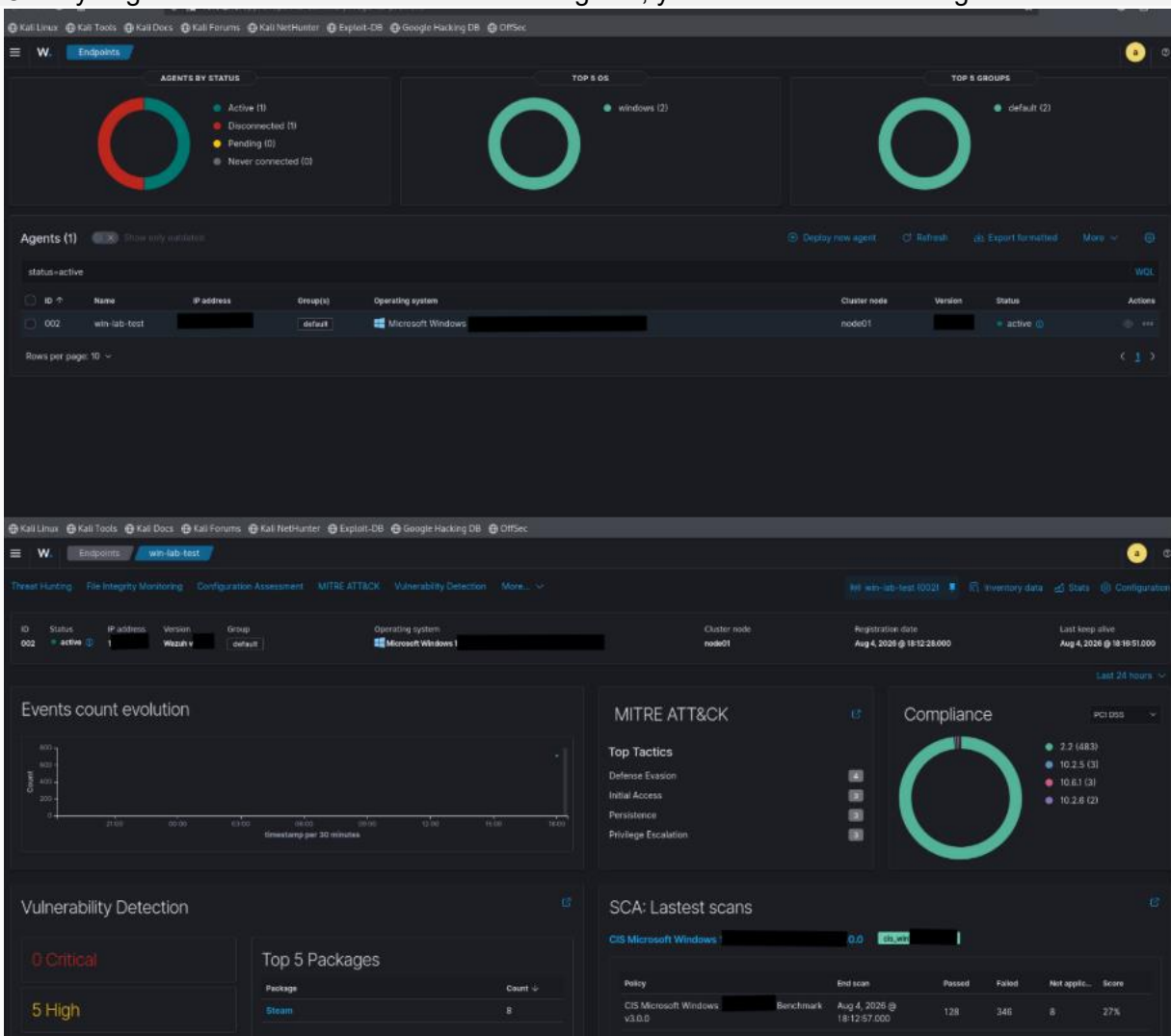
(kali@kali)-[~/asp-compose]
$ sudo /var/ossec/bin/agent control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: kali (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: test-win-laptop1, IP: any, Disconnected
  ID: 002, Name: win-lab-test, IP: any, Active

List of agentless devices:

```

Once you go to Wazuh Dashboard>End Point Agents, you should see something like this:

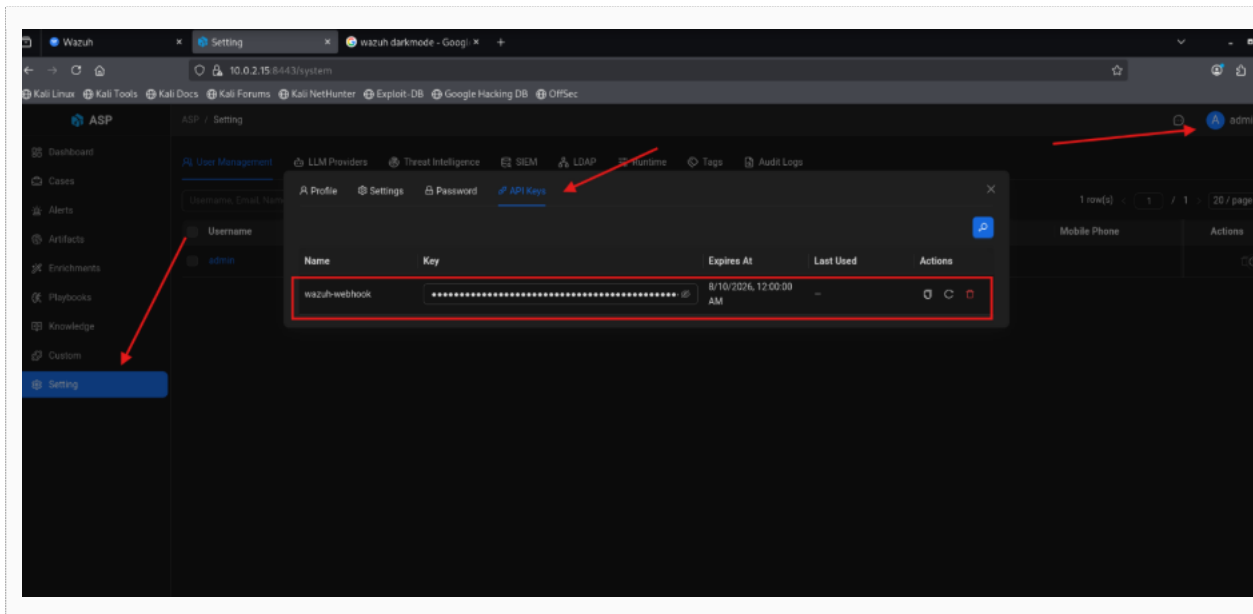


PART B.6 — Bridge Wazuh → ASP (the webhook)

Step 11 — Get the ASP webhook API key

There is NO webhook token in .env in v0.5.2. Generate a per-user API key in the UI:

8. Top-right account menu (your admin avatar) → API Keys.
9. Create a key named e.g. 'wazuh-webhook', set a far-out expiry.
10. Copy the key immediately (shown once). This is the <api_key> for ossec.conf.



Step 12 — Install the custom-asp integration script

Drag the shipped custom-asp onto the Kali desktop, then install it into Wazuh with correct ownership/permissions:

```
sudo cp ~/Desktop/custom-asp /var/ossec/integrations/custom-asp
sudo chmod 750 /var/ossec/integrations/custom-asp
sudo chown root:wazuh /var/ossec/integrations/custom-asp
sudo dos2unix /var/ossec/integrations/custom-asp 2>/dev/null
```

WHAT THE SCRIPT DOES

Wazuh calls: `custom-asp <alert_file> <api_key> <hook_url>`. The script reads the alert JSON, maps `rule.level` → severity (`>=12` Critical, `>=8` High, `>=5` Medium, else Low), pulls `srcip/user` as IOCs, and POSTs a Kibana-shaped envelope `{ rule.name = 'Wazuh-Alerts', context.hits[0]._source = <mapped alert> }` to your `hook_url` with `'Authorization: Bearer <api_key>'`. No pip installs needed. It logs each send to `/var/ossec/logs/integrations.log`.

Step 13 — Register the hook in ossec.conf (exact placement)

Open the config in nano:

```
sudo nano /var/ossec/etc/ossec.conf
```

Jump to the closing tag: press Ctrl+W, then Ctrl+V (this goes to the end of the file). Find the line:

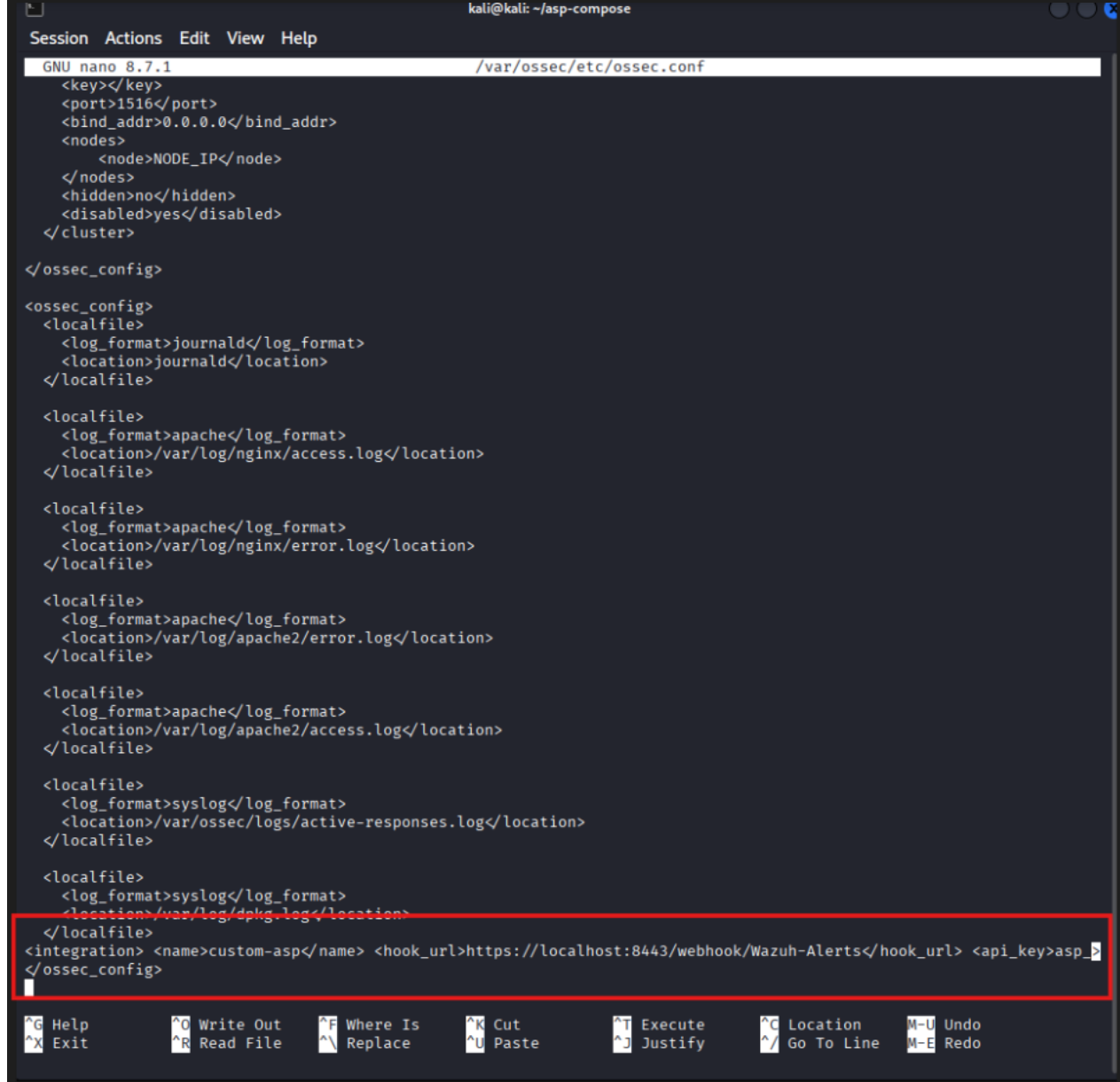
```
</ossec_config>
```

Add this block on a blank line JUST BEFORE that `</ossec_config>`, pasting your real API key between the `<api_key>` tags:

```

<integration>
  <name>custom-asp</name>
  <hook_url>https://localhost:8443/api/webhook/kibana/</hook_url>
  <api_key>PASTE_YOUR_ASP_API_KEY_HERE</api_key>
  <level>7</level>
  <alert_format>json</alert_format>
</integration>

```



```

GNU nano 8.7.1 /var/ossec/etc/ossec.conf
<key></key>
<port>1516</port>
<bind_addr>0.0.0.0</bind_addr>
<nodes>
  <node>NODE_IP</node>
</nodes>
<hidden>no</hidden>
<disabled>yes</disabled>
</cluster>

</ossec_config>

<ossec_config>
  <localfile>
    <log_format>journald</log_format>
    <location>journald</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/nginx/access.log</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/nginx/error.log</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/apache2/error.log</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/apache2/access.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/dpkg.log</location>
  </localfile>
  <integration> <name>custom-asp</name> <hook_url>https://localhost:8443/webhook/Wazuh-Alerts</hook_url> <api_key>asp_
</ossec_config>

```

^G Help ^O Write Out ^F Where Is ^R Cut ^T Execute ^C Location M-U Undo
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

TWO THINGS PEOPLE GET WRONG HERE

(1) The tag is `<api_key>` — the KEY goes BETWEEN the tags, not inside the tag name. (2) The `hook_url` MUST be `/api/webhook/kibana/` (with the `/api/` prefix and trailing slash). The old `/webhook/Wazuh-Alerts` path returns 405 Not Allowed in v0.5.2. Since Wazuh and ASP are on the same box, `localhost:8443` over loopback is perfect — no IP needed.

Save and exit nano: `Ctrl+O` → `Enter` → `Ctrl+X`. Then restart the manager and watch the log:

```

sudo systemctl restart wazuh-manager
sudo tail -f /var/ossec/logs/integrations.log

```


A successful push logs: custom-asp: SENT <alert> -> <https://localhost:8443/api/webhook/kibana/>
HTTP 200

PART C — Prove It End-to-End

Step 14 — Generate test telemetry (light pentest on Windows)

On the Windows machine (Admin PowerShell), run one of these. These are self-contained lab actions that trip Wazuh rules at/above level 7. Keep the Kali integrations.log tail running while you do it.

Type this on Powershell that you opened as admin line by line.

Trigger A — Account create / privilege change (RELIABLE, out-of-the-box)

```
net user labbackdoor P@ssw0rd123! /add
net localgroup administrators labbackdoor /add
net localgroup administrators labbackdoor /delete
net user labbackdoor /delete
```

Trigger B — Recon / discovery burst

```
whoami /all
net user
net localgroup administrators
ipconfig /all
netstat -ano
Tasklist
Systeminfo
```

HONEST CAVEAT

Trigger A (account/group) and Trigger B (recon/rootcheck/netstat) fire out-of-the-box on a default Windows agent — start with those. Trigger C (PowerShell/logon detail) needs Sysmon or Script-Block-Logging forwarded to Wazuh, or it may be silent. Check with: `Get-Service sysmon* -ErrorAction SilentlyContinue`

```

Session Actions Edit View Help
$ sudo cp ~/Desktop/custom-asp /var/ossec/integrations/custom-asp
(kali@kali)-[~/asp-compose]
$ sudo chmod 750 /var/ossec/integrations/custom-asp
(kali@kali)-[~/asp-compose]
$ sudo chown root:wazuh /var/ossec/integrations/custom-asp
(kali@kali)-[~/asp-compose]
$ sudo dos2unix /var/ossec/integrations/custom-asp 2>/dev/null
(kali@kali)-[~/asp-compose]
$ sudo nano /var/ossec/etc/ossec.conf
(kali@kali)-[~/asp-compose]
$ sudo systemctl restart wazuh-manager
(kali@kali)-[~/asp-compose]
$ sudo tail -f /var/ossec/logs/integrations.log
2026-08-04 18:39:50 custom-asp: HTTPError 405 from https://localhost:8443/webhook/Wazuh-Alerts : b'<html>\r\n<head><title>405 Not Allowed</title></head>\r\n<body>\r\n<center><h1>405 Not Allowed</h1></center>\r\n<hr><center>nginx/1.31.3</center>\r\n</body>\r\n</html>\r\n'
2026-08-04 18:39:50 custom-asp: HTTPError 405 from https://localhost:8443/webhook/Wazuh-Alerts : b'<html>\r\n<head><title>405 Not Allowed</title></head>\r\n<body>\r\n<center><h1>405 Not Allowed</h1></center>\r\n<hr><center>nginx/1.31.3</center>\r\n</body>\r\n</html>\r\n'
2026-08-04 18:39:54 custom-asp: HTTPError 405 from https://localhost:8443/webhook/Wazuh-Alerts : b'<html>\r\n<head><title>405 Not Allowed</title></head>\r\n<body>\r\n<center><h1>405 Not Allowed</h1></center>\r\n<hr><center>nginx/1.31.3</center>\r\n</body>\r\n</html>\r\n'
2026-08-04 18:50:34 custom-asp: SENT Host-based anomaly detection event (rootcheck). → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:50:34 custom-asp: SENT Host-based anomaly detection event (rootcheck). → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:50:34 custom-asp: SENT Host-based anomaly detection event (rootcheck). → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:50:34 custom-asp: SENT Host-based anomaly detection event (rootcheck). → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:50:34 custom-asp: SENT Host-based anomaly detection event (rootcheck). → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:50:38 custom-asp: SENT Listened ports status (netstat) changed (new port opened or closed). → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:55:56 custom-asp: SENT User account enabled or created → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:55:56 custom-asp: SENT User account enabled or created → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:55:56 custom-asp: SENT User account changed → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:56:01 custom-asp: SENT Administrators Group Changed → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:56:07 custom-asp: SENT Administrators Group Changed → https://localhost:8443/api/webhook/kibana/ HTTP 200
2026-08-04 18:56:13 custom-asp: SENT User account disabled or deleted → https://localhost:8443/api/webhook/kibana/ HTTP 200

```

Step 15 — Watch the Case appear (with a Gemini verdict)

Give it a few seconds, then open the ASP UI → Cases. New Cases appear as alerts arrive (live, one-by-one). Open the newest → you should see a rich Description and a populated Verdict (AI), Severity (AI), Confidence (AI), Impact (AI).

The top screenshot shows the Wazuh ASP 'Cases' dashboard. The left sidebar contains navigation links: Dashboard, Cases, Alerts, Artifacts, Enrichments, Playbooks, Knowledge, Custom, and Setting. The 'Cases' link is highlighted. The main area displays a table of cases with columns: Case ID, Title, Status, Alerts, Playbooks, Enrichments, Category, Severity, Severity (AI), and Actions. A red arrow points to the 'Cases' link in the sidebar, and another red arrow points to the case entry for CASE_000011.

Case ID	Title	Status	Alerts	Playbooks	Enrichments	Category	Severity	Severity (AI)	Actions
CASE_000012	[High] User account disabled or deleted on win-lab-test	New	View (2)	View (2)	View (6)	-	High	Low	
CASE_000011	[Critical] Administrators Group Changed on win-lab-test	New	View (6)	View (6)	View (6)	-	Critical	Medium	
CASE_000016	[High] User account changed on win-lab-test	New	View (2)	View (2)	View (6)	-	High	Medium	
CASE_000009	[High] User account enabled or created on win-lab-test	New	View (2)	View (2)	View (6)	-	High	Low	
CASE_000008	[Medium] Listener ports status (netstat) changed (new port opened or cl...	New	View (1)	View (2)	View (6)	-	Medium	-	
CASE_000007	[Medium] Host-based anomaly detection event (rootcheck) on kali	New	View (2)	View (2)	View (6)	-	Medium	Medium	

The bottom screenshot shows the detailed view of the case '[Critical] Administrators Group Changed on win-lab-test'. The left sidebar contains navigation links: Case ID, Severity, Severity (AI), Category, Description, First Seen Alert, Detection Time (TTR), Acknowledged Time (TTR), Resolved & Close, Closed Time (TTR), and Verdict (AI). The 'Severity' link is highlighted. The main area displays the case details, including the title, status, confidence, impact, priority, and various time metrics. Red arrows point to the 'Severity' link in the sidebar, the 'Severity (AI)' field, the 'Confidence (AI)' field, the 'Impact (AI)' field, the 'Priority (AI)' field, and the 'Verdict (AI)' field.

[Critical] Administrators Group Changed on win-lab-test
Created 16 minutes ago · Updated 7 minutes ago

Case ID: CASE_000011
Status: New
Assignee: -

Severity: Critical
Severity (AI): Medium
Confidence: -
Confidence (AI): Medium
Impact: -
Impact (AI): Low
Priority: -
Priority (AI): Medium

Category: -
Correlation UID: wazuh-00154-win-lab-test
Tags: wazuh, T1484

Description: Administrators Group Changed i

First Seen Alert: 8/4/2026, 6:56:00 PM
Detection Time (TTR): 26m 57s

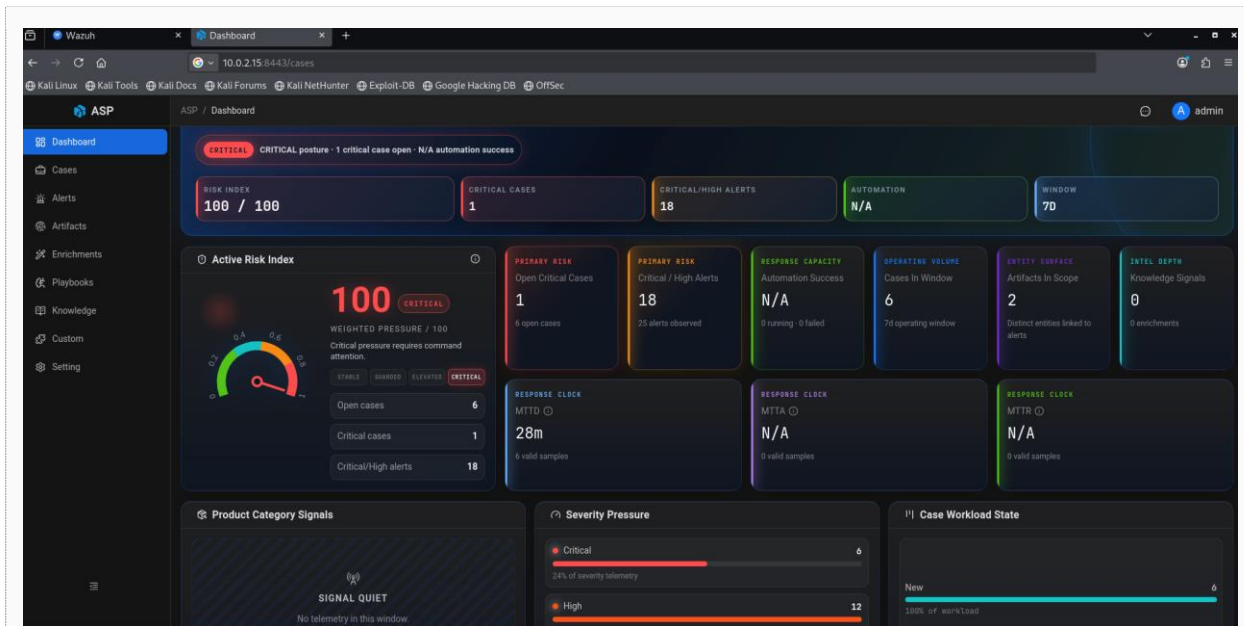
In Process & On Hold

Acknowledged Time (TTR): -
Time to Acknowledge (TTA): -

Resolved & Close

Closed Time (TTR): -
Time to Respond (TTR): -
Verdict: -
Verdict (AI): Suspicious

Finally, they main dashboard may look like this:



DIAGNOSTIC — IF NO CASE APPEARS (splits the problem in one command)

Check the stream depth inside the Redis container (DB 1):

```
docker exec asp-compose-redis-stack-1 redis-cli \
  -a "$(grep REDIS_PASSWORD /home/kali/asp-compose/.env | cut -d= -f2)" \
  -n 1 XLEN Wazuh-Alerts
```

- XLEN = 0 → the alert never arrived → problem is the webhook / ossec.conf (Steps 12–13): re-check hook_url, api_key, and integrations.log.

- XLEN > 0 but no Case → the alert arrived but the Module errored → read the exact reason:

```
docker compose logs asp-worker-module --tail 40
```

A Python traceback there names the failing field/method. If you ever edit the Module, replay the queued alerts through the new code with:

```
docker exec asp-compose-redis-stack-1 redis-cli \
  -a "$(grep REDIS_PASSWORD /home/kali/asp-compose/.env | cut -d= -f2)" \
  -n 1 XGROUP SETID Wazuh-Alerts agentic-modules 0
docker compose restart asp-worker-module
```

NOTE ON 'Insufficient Data' VERDICTS

If a verdict says 'Insufficient Data', the alert was genuinely thin (e.g. a bare 'account created' with almost no context). The shipped Module already enriches the description with rule id+level, user, IPs, Windows Event ID, MITRE and the raw log to avoid this. Higher-signal events (failed logons, recon bursts) produce the most substantive verdicts.

Common issues

Symptom	Likely cause → fix
405 Not Allowed on POST	Wrong path → hook_url must be /api/webhook/kibana/ (not /webhook/<stream>).
401 Unauthorized	Wrong/missing key → match <api_key> in ossec.conf to the ASP API key from Step 11.
Test times out then works	gemini-3.1-flash-lite first-token latency → just retry Test; config is fine.

Symptom	Likely cause → fix
XLEN grows, no Case	Module errored → docker compose logs asp-worker-module --tail 40; fix field/enum, replay.
Verdict = Insufficient Data	Sparse alert → use higher-signal triggers; description enrichment already applied.
Nothing in integrations.log	Alert below <level>7</level>, or you didn't restart wazuh-manager after editing ossec.conf.

PART D — Clean Teardown (leave no residue)

FASTEST PATH

Snapshotted in Part 0? Just power off → Restore Snapshot 'Before-Lab12-ASP'. Everything below is the manual, verifiable version.

D.1 Remove the Wazuh integration

```
sudo rm -f /var/ossec/integrations/custom-asp
sudo nano /var/ossec/etc/ossec.conf # delete the <integration>...custom-asp...</integration>
block
sudo systemctl restart wazuh-manager
```

D.2 Tear down ASP containers + data

```
cd ~/asp-compose && docker compose down -v --remove-orphans
docker rmi $(docker images 'ghcr.io/funnywolf/*' -q) 2>/dev/null
cd ~ && rm -rf ~/asp-compose ~/asp-compose.tar.gz
```

D.3 Kill the credentials & lab user

- Google AI Studio → API keys → delete the Gemini key from Step 7.
- ASP account → API Keys → delete the 'wazuh-webhook' key from Step 11 (moot after D.2, but do it if ASP is kept).
- On Windows, ensure the lab user is gone: net user labbackdoor /delete (Trigger A already deletes it).

D.4 Close ports the lab opened & sweep for residue

```
sudo ss -tlnp | grep -E '':8443|:8001|:9001' || echo 'clean: no ASP lab ports listening'
docker ps -a; docker volume ls; docker network ls
ls -la /var/ossec/integrations/ | grep -i asp || echo 'no asp integration left'
ls -la ~ | grep -i asp || echo 'no asp artifacts in home'
```

Optional: remove the Host-Only adapter (VirtualBox → Settings → Network → Adapter 2 → untick Enable) to return networking to NAT-only.

D.5 (Optional) Remove Docker if installed only for this lab

```
sudo systemctl disable --now docker
sudo apt purge -y docker.io
sudo rm -rf /var/lib/docker /var/lib/containerd /usr/local/lib/docker/cli-plugins/docker-
compose
```

Appendix A — Installing Wazuh Server on Linux

If the Kali (or Linux) box does not already have Wazuh, install the all-in-one Wazuh server (manager + indexer + dashboard) BEFORE Part B. This is the official quick path; the full step-by-step is linked at the bottom.

HEADS-UP FOR THIS LAB

The Wazuh dashboard binds to 443 by default — that is exactly why we run ASP on 8443 (Step 4). If you install Wazuh AFTER ASP, expect the 443 owner to be the dashboard.

A.1 Assisted (all-in-one) installation

Run as root on a clean, internet-connected Linux host with 2 CPU / 4 GB RAM minimum:

```
curl -sO https://packages.wazuh.com/4.12/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

When it finishes it prints the admin credentials, e.g.:

```
User: admin
Password: <auto-generated>
```

Retrieve the password later if needed:

```
sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
```

A.2 Verify the services

```
sudo systemctl status wazuh-manager --no-pager
sudo systemctl status wazuh-indexer --no-pager
sudo systemctl status wazuh-dashboard --no-pager
sudo /var/ossec/bin/wazuh-control status    # want wazuh-remoted + wazuh-apid running
sudo ss -tlnp | grep -E '1514|1515'       # agent comms + enrollment listening
```

A.3 Access the dashboard

```
# https://<kali-ip>/      (default 443; accept the self-signed cert; log in as admin)
```

IF THE API SAYS 'No API available'

Right after a reboot the manager/API may still be starting (you'll see 'wazuh-remoted->failed' or a 3099 error). Wait ~60s, or run: `sudo systemctl restart wazuh-manager`, then re-check the dashboard's API connection. On an 8 GB box running Wazuh + 13 ASP containers this can happen under memory pressure — if it persists, free RAM with `'cd ~/asp-compose && docker compose stop'` while Wazuh stabilises, then `'docker compose start'`.

Official step-by-step reference:

- <https://documentation.wazuh.com/current/installation-guide/wazuh-server/step-by-step.html>

Appendix B — Screenshot Checklist

Shot	Capture when...
S01	docker --version + docker compose version succeed
S02	~/asp-compose extracted
S03	init.sh finished (migrations OK; 443 bind error noted)
S04	ASP_HTTPS_PORT=8443 set + docker compose up -d shows 13/13 healthy
S05a	docker compose ps healthy + doctor.sh green
S05b	ASP login page on :8443
S06	Dashboard after admin login
S07	Gemini LLM provider saved + Enabled
S08	'Wazuh Alerts' Module loaded (Stream = Wazuh-Alerts)
S09	Windows agent Active in agent_control -l / Wazuh Endpoints
S10a	ASP API Keys: 'wazuh-webhook' key created
S10b	integrations.log SENT ... /api/webhook/kibana/ HTTP 200
S11	Pentest events flowing (HTTP 200s)
S12	Case with artifacts + Gemini verdict (hero shot)
S13	Residue sweep clean + keys revoked

Appendix C — Files Shipped With This Guide

File	Put it here	Role
custom-asp	/var/ossec/integrations/custom-asp (chmod 750, chown root:wazuh)	Wazuh → ASP webhook push (Kibana masquerade). Reads api_key + hook_url from ossec.conf.
wazuh_alerts_module.py	<asp-compose>/custom/modules/	ASP Module: consumes 'Wazuh-Alerts' stream → Case/Alert/Artifact + schedules Gemini verdict.

HANDOVER NOTE

Both files are the final, tested versions. Do not retype them from screenshots — ship the actual files. The Module encodes several ASP v0.5.2-specific field names and enums that are easy to get wrong by hand.

Appendix D — Source References

- Repo: <https://github.com/FunnyWolf/agentic-soc-platform>
- ASP deployment (Compose bundle): <https://asp.viperrtp.com/asp/quick-start/deployment/>
- ASP alert ingestion (webhook + stream-name contract): <https://asp.viperrtp.com/asp/development/alert-ingestion/>
- ASP Kibana webhook (the /api/webhook/kibana/ route we use): <https://asp.viperrtp.com/asp/development/alert-ingestion/kibana-webhook/>

- ASP module development (BaseModule, create_alert_with_context):
<https://asp.viperrtp.com/asp/development/module-examples/>
- Gemini OpenAI-compatibility: <https://ai.google.dev/gemini-api/docs/openai>
- Wazuh server step-by-step install: <https://documentation.wazuh.com/current/installation-guide/wazuh-server/step-by-step.html>